

FROM THE BLACKTOP TO BOTH STORES.

Why the game freezes and crashes on iPhone and iPad, every bug that has to die before launch, the dead weight slowing it down, and a phased plan to package it for the App Store and Google Play as a game people will pay for monthly.

5

PO GAME-STOPPERS

22

P1 MUST-FIX BEFORE
STORE

162

MB OF DEAD ASSETS

20

WEEKS TO STORE
LAUNCH

Prepared

2026-09-12 · read-only audit, no
code changed

Production

main @ 6bbfa8f · street-
kickball-kings.vercel.app

Working branch

feat/switch-cinematic-crown-
pitch (uncommitted)

READ THIS PAGE IF NOTHING ELSE

IT IS NOT A MILLION BUGS. IT IS FIVE BIG ONES AND A DIET.

The engine underneath is in better shape than it feels from a phone. The test suite passes (688 tests, 65 files), the frame loop survives thrown errors, and the gameplay watchdogs added in July still hold. What broke the phone experience is a small number of structural problems that all land at once on iOS, plus a launch bar the project has not been built toward yet.

Every game

that ends on a play has hung at the final out since Aug 28. The game-over poll waits for a phase the scene never leaves.

~750 MB

estimated live memory in a match on iPhone. Textures alone measure 267 MB. iOS kills web content around 1 GB on 4 GB phones.

628 MB

of assets in the project, 162 MB never referenced. Two 18 MB videos stream at 15 Mbps per city, at once.

- 1 THE FREEZE HAS A SINGLE CAUSE ON PRODUCTION RIGHT NOW**

Since the rules pass shipped on Aug 28, `fireMatchOver` re-polls while the phase is RESOLVE, and after a game-ending play nothing ever changes the phase. Final out, walk-off run, walk-off homer: the score updates, then nothing. No victory lap, no box score, no button. This is on main and no harness drives a game to its end.
- 2 THE CRASH AND RELOAD IS A MEMORY BUDGET PROBLEM, NOT A BUG**

Sixteen recoloured character atlases (seven at 2048², 21 MB each on the GPU), two 2-megapixel H.264 videos decoding per field, a 4× MSAA half-float post chain at 2× pixel ratio, a 2048² soft shadow map, and every decoded music track cached for the life of the tab. Nothing is freed between matches: the field and its two videos are never disposed. Match two is heavier than match one.
- 3 WHEN SOMETHING FAILS, THE GAME HAS NO WAY TO SAY SO**

Match build has no error path (a failed GLB or video fetch leaves a black screen forever), there is no global error or crash reporter, the main canvas has no WebGL context-loss handler, and the audio bus does not know about iOS's *interrupted* state. On Android these are rare. On iOS they are the everyday failure modes.
- 4 THE GAME PROMISES PROGRESSION IT DOES NOT HAVE**

The menu shows XP, crowns, a win streak and a daily challenge. None of them are ever written. Derby says coming soon. There are no accounts, no cloud save, no leaderboards, no seasons. A subscription needs these to exist and to be real.
- 5 THE STORE PATH IS REALISTIC, AND IT IS A WRAPPER, NOT A REWRITE**

Twenty-one thousand lines of working JavaScript and three.js package into iOS and Android apps with Capacitor. The work is the asset diet (installed size must drop from 500+ MB to under 150 MB), in-app purchase rails, accounts, and the retention loops that justify a monthly price. Estimated twenty weeks with a soft launch gate at week sixteen.

THREE DIFFERENT FAILURES WEAR THE SAME FACE

"It froze" on an iPhone is three separate things: the game logic stops advancing (a freeze), Safari or the PWA runs out of memory and reloads the page (a crash), or the page is alive but shows black or goes silent (a dead render or audio path). Each has its own root cause and its own fix. Root causes were verified by reading the code, reproducing on production in a WebKit iPhone profile, and measuring GPU memory in a live match.

A. THE FREEZE: STATE-MACHINE DEADLOCKS

P0 GAME OVER NEVER FIRES SRC/GAME/MATCHSCENE.JS:647

The poll that waits for the victory lap bails while `phase` is PITCH, KICK_ANIM, LIVE or RESOLVE. Every play resolver sets RESOLVE and then, on GAME_END, returns without calling `nextAtBat`, and `nextAtBat` itself exits on GAME_END before it would set SETUP. Sixteen phase writers were checked; none run after a game-ending play. Added in commit 9429882 (Aug 28), merged in PR #111, live on production. A caught-stealing final out and a ball-four walk-off deadlock the same way from the PITCH phase.

What the player sees: final out or walk-off resolves, score bug is right, camera settles, and the game sits there forever. Force-quit is the only exit. On Android this is identical; it feels like an Apple problem because that is where the dev's testers are.

- **P1** The 14-second net and the settled-field net run *after* the runner, duel and defense code in the same frame. A per-frame exception in that code (the class of bug behind July's taken-pitch freeze) skips both nets every frame, so the loop survives but the play never closes. `matchScene.js:4839-4907`
- **P1** The new side-switch cinematic on the working branch marks the request `handled` before the beat exists and holds the at-bat behind a step clock with no scene-side timeout. `src/cinematics/director.js:52`, `matchScene.js:1166`
- **P1** `throwing` is set before the release and has no `finally`; one exception in a throw disables tags, after-throw and duel closure for the rest of the play. `matchScene.js:3569`
- **P2** No dead-man timer covers SETUP, PITCH, KICK_ANIM or FOUL; each relies on a single scheduled timer. This is where the next silent stall will land.

B. THE CRASH: THE MEMORY BUDGET

Measured in a live production match with a WebKit iPhone 13 profile. Textures are device-independent; render targets scale with screen pixels (a real iPhone 13 canvas is 1.4× larger than the probe's). Everything else is an estimate from file sizes and decoder behaviour, labelled as such.

IPHONE SE / XR / 11 KILL ZONE ≈ 600-900 MB



■ Character atlases + decal textures (measured 242–267 MB)

■ Post-chain render targets, 4× MSAA half-float (measured 27–69 MB, ~100 MB at true iPhone resolution) ■ 2048² shadow map (16 MB)

■ Two 1248×1664 H.264 decoders + video textures (est.)

■ Decoded WebAudio buffers held forever: theme, city track, beat, ambience, 34 SFX, announcer lines (est.)

■ JS heap: 16 rigs, 58 skinned meshes, mocap packs, three.js (est.) ■ CPU-side recolour canvases backing the live textures (est.)

Apple does not publish the number, but WebKit's web content process on a 4 GB phone is jetsammed somewhere between 600 MB and roughly 1 GB, and lower when other apps are

open. This match sits inside that band on the first game and grows on the second, because the previous field, its two video decoders and six bus subscriptions are never released.

B. THE CRASH, CONTINUED: WHAT IS FILLING THE BUDGET

SOURCE	MEASURED / EVIDENCE	WHY IT MATTERS ON IOS
Seven 2048 ² atlases	6 legacy archetypes + monarchs-23 ship a 2048 ² PNG inside the GLB (6–8 MB each); the other 14 archetypes are 1024 ² WebP at 60–130 KB. Each 2048 ² atlas costs 21 MB on the GPU with mips. <code>public/assets/models/archetypes</code>	149 MB of the 267 MB texture budget is seven characters.
Recolour per (archetype, hex, skin tone)	Cache key includes skin tone, so casts defeat sharing; 16 canvases per match, 16-entry LRU. <code>src/game/glbCharacters.js:47-119</code>	Canvas backing stores stay on the CPU as long as the texture is live.
Two backdrop videos per field	1248×1664 H.264 at 6–15 Mbps (block-party: 14.6 Mbps × 2, 18 MB each) for a 10-second loop; both play at once; re-uploaded as textures every frame. <code>src/game/field.js:272-381, fields.json</code>	Two hardware decoders plus 17 MB of <code>texImage2D</code> per frame. WebKit's decoder already "lies" (July postmortem); two of them doubles the odds.
Post chain	Composer target 4× MSAA HalfFloat, cloned twice, plus 10 bloom mip targets, at 2× DPR; an unused GTAO pass is still constructed. <code>src/engine/renderer.js:74-155</code>	~100 MB of render targets at iPhone resolution and a full-screen 11-pass bloom for a 0.18-strength glow.
Decoded audio never evicted	<code>buffer()</code> caches every decoded <code>AudioBuffer</code> in a Map with no cap. A 2–3 minute city track decodes to 40–60 MB of PCM. <code>src/engine/audio.js:245-255</code>	Theme + city track + beat + ambience + SFX + booth lines accumulate across matches.
Never disposed between matches	<code>destroy()</code> only removes the field from the scene; no geometry, material, texture or video is disposed; videos are never paused; six <code>bus.on</code> handlers keep firing into a dead HUD. <code>matchScene.js:4977-5019, field.js</code>	Rematch and Run the Map stack a second field on top of the first.

C. THE DEAD SCREEN: NO ERROR PATH, NO RECOVERY

- P0** Match build cannot fail gracefully. `startMatchFlow` awaits two GLB team builds and three videos with no try/catch and no retry; a dropped cellular fetch or a partial service-worker response leaves the black cover div on screen forever. `src/main.js:298-343`
- P1** No global error handler or crash reporter. There is no `window.onerror`, no `unhandledrejection` listener and no telemetry. The dev learns about crashes from players. `src/main.js`
- P1** Main canvas ignores WebGL context loss. iOS drops contexts under memory pressure and on backgrounding. The Locker preview handles `webglcontextlost`; the game canvas does not, so the loop keeps rendering into a dead context: a black game with a live HUD. `src/engine/renderer.js, src/ui/lockerPreview.js:32`
- P1** The city music is a container Safari's decoder rejects. All twelve in-match tracks are fragmented MP4 (2,585 `moof` boxes, zero duration in the header). Chrome decodes them; Safari's `CoreAudio` path needs a sample table and throws. The failure is cached as silence for the session, which matches the dev's own note that city music "isn't working". `public/assets/audio/music, src/engine/audio.js:245-255`
- P1** The build needs iOS 16.4 or newer and says nothing below it. Vite's default target emits class static blocks and private fields; iPhone 7, iPad Air 2 and any un-updated 16.0–16.3 device throw a parse error and show an empty stage. `vite.config.js, index.html`
- P1** Two WebGL contexts at 2× pixel ratio. The Locker spins its own renderer and rAF loop while the main engine keeps rendering the full post chain behind the menu. iOS caps live contexts and this doubles GPU memory on the menu screens. `src/ui/lockerPreview.js:15, renderer.js:277-317`
- P1** Audio does not know iOS's interrupted state. After a phone call, Siri or a Control Center swipe, WebKit reports `state === 'interrupted'`; the bus only resumes on `'suspended'`. The game goes silent until a reload. `src/engine/audio.js:213`
- P1** 14-second main-thread stall during character build. Measured on production: sixteen serial recolour passes of 130–400 ms each block the UI thread while the intro videos play; TAP TO SKIP does not respond. `src/game/glbCharacters.js:44-93, main.js:298`

BUG LEDGER

P0 stops the game or kills the tab. P1 corrupts state, leaks, or blocks a store submission. P2 is visible polish. IDs are stable so they can become issues. Items marked **branch** live in the uncommitted switch-cinematic work, not on production.

ID	SEV	BUG	WHERE	FIX DIRECTION
B01	P0	Game-over poll waits on a phase the scene never leaves; every game ending on a play hangs.	matchScene.js:647-648	Gate on cinematicLock and a live ball only; write a terminal GAME_OVER phase; cap the poll.
B02	P0	Per-match GPU + audio memory (~750 MB est.) sits in the iOS jetsam band; grows each match.	glbCharacters.js, field.js, audio.js, renderer.js	Asset diet + device tier + dispose on destroy (see §04).
B03	P0	Match build has no error path; a failed asset fetch leaves a black screen forever.	main.js:298-343	try/catch around the build with a retry screen and a loading indicator.
B04	P0	Field, ball, two video decoders, the 2048² shadow map, the scene environment map and six bus subscriptions are never released between matches.	matchScene.js:251-274, 4990; field.js:272-311; renderer.js:221	Add dispose() to the field (pause + unload videos, sun.dispose()); dispose the outgoing env map; call every unsubscribe in destroy().
B05	P0 branch	Switch cinematic flags handled before it runs and can hold SETUP forever if a step throws.	director.js:52, 99-101; matchScene.js:1166	Set handled after run(); add a scene-side timeout; check duration before onUpdate.
B06	P1	14 s net and settled-field net sit below the code they guard; a per-frame throw starves both.	matchScene.js:4839-4907, 2526-2541	Hoist both nets above the phase blocks, as the RunnerWatchdog already is.
B07	P1	Two runners settling one bag silently drops a runner from the books; no out recorded.	matchScene.js:2579-2592, 2413-2424, 3067	Deterministic collision rule at finalize; occupancy check in the stall rule and force-settle.
B08	P1	Pickle clock counts from the kick, not the rundown; the net can award the trapped runner the forward bag.	matchScene.js:4897, 3061-3068	Duel-local timer; settle a trapped runner back or out, never forward.
B09	P1	throwing has no finally; one exception disables tags and duel closure for the play.	matchScene.js:3569-3667	try/finally around the release; validate base at entry.
B10	P1	No global error handler, no crash or performance telemetry.	main.js	Sentry (or equivalent) with release tags; onerror + unhandledrejection.
B11	P1	Main canvas has no webglcontextlost / restored handling.	renderer.js	Handle both; pause the loop, rebuild the composer, show a "tap to resume" card.
B12	P1	Audio bus ignores WebKit's interrupted context state; silence after a call or Control Center.	audio.js:213	Resume on interrupted too, on the next user gesture and on visibilitychange.
B13	P1	Decoded AudioBuffers cached forever; music tracks decode to 40–60 MB each.	audio.js:245-255	Evict music buffers on track change; keep only SFX resident.
B14	P1	music() race: theme restarts over the intro video after stopMusic().	audio.js:264-275	Token-guard the async play against a later stop.
B15	P1	Serial character build blocks the main thread in 130–400 ms chunks; 14 s stall measured.	glbCharacters.js:44-93, 836-860; main.js:298	Parallel fetch, chunk pixel work across frames or a Worker, quantise skin tones for cache hits.

ID	SEV	BUG	WHERE	FIX DIRECTION
B16	P1	Post chain: 4× MSAA HalfFloat ×2, bloom, grade, and an unused GTAO pass, at 2× DPR, always on, even behind menus.	renderer.js:74-155, 277-317	Device tier at boot; skip <code>composer.render()</code> under DOM screens; delete GTOA.
B17	P1	2048² PCFSOFT shadow map with 16 un-culled, shadow-casting 25k-triangle rigs.	renderer.js:75; field.js:573; glbCharacters.js:617	1024² PCF, frustum culling from a rig bounding sphere, cast shadows only near the camera.
B18	P1	Two 1248×1664 H.264 backdrops per field at up to 15 Mbps each, both decoding at once.	field.js:272-381; public/assets/video	One 720×960 video ≤ 3 Mbps per field; poster only on low tier.
B19	P1	Service-worker cache is unbounded; a stale index can reference deleted hashed chunks offline.	public/sw.js	Precache the app shell only; cap and evict media; version the cache per deploy.
B20	P1	Cinematic lock has two owners (director and replay); the first to finish unlocks both and the replay teleports everyone back.	matchScene.js:251-252; director.js:64; replay.js:119-163	Reference-count the lock; have <code>run()</code> finish an active replay.

03 THE BUG LEDGER

CONTINUED

ID	SEV	BUG	WHERE	FIX DIRECTION
B21	P1	Coin-toss timers outlive navigation and can call <code>beginMatch</code> after the player left the screen.	screens.js:349-372; router.js:16	Give screens an <code>unmount</code> that clears their timers.
B22	P1	HUD rebuilds runner alerts and re-queries the diamond every LIVE frame; two forced layout reads per frame after style writes.	hud.js:212, 658; matchScene.js:3938, 3987	Cache nodes and the canvas rect; dedupe setters by key.
B23	P1	Boot: one 1.19 MB JS bundle, render-blocking Google Fonts import, PMREM built before any 3D screen.	vite.config.js; ui.css:8; renderer.js:90	Code-split Locker, tutorial, director; self-host fonts; defer PMREM.
B24	P1	iOS Safari (non-PWA): stage is <code>100vh</code> , so bottom HUD buttons sit under the toolbar.	ui.css:67	<code>100dvh</code> with a <code>100vh</code> fallback.
B25	P2	Tags and pad throws still fire after the books close; a SAFE call can be followed by TAGGED OUT.	matchScene.js:2364-2398, 3547, 4127	Guard on <code>!playFinalized</code> ; hide the pad in <code>finalizePlay</code> .
B26	P2	<code>tryNext</code> waits on <code>ball.mode === 'flying'</code> with no cap; a NaN velocity polls forever.	matchScene.js:2619-2624; ball.js:151-176	Cap at 4 s and re-place the ball at the mound.
B27	P2	<code>engine.paused</code> has two writers; the inning card's 6 s timeout un-pauses under the pause overlay.	matchScene.js:718; main.js:221-232	Reference-count pause.
B28	P2	Skipped set-piece videos keep downloading and decoding after the wrapper is removed.	videoPlayer.js:21-34	Pause, clear <code>src</code> , <code>load()</code> in <code>finish</code> .
B29	P2	AI fielding decisions can early-return with nothing scheduled; the play closes only via the 8–14 s nets.	matchScene.js:3445-3476	Schedule <code>ballControlled</code> after ~1 s on early return.
B30	P2	HUD <code>destroy()</code> leaves ~8 timers pointing at the removed element.	hud.js:420-846	Track timer ids; clear in <code>destroy()</code> .
B31	P2	Menu shows XP, crowns, streak and a daily challenge that are never written; Derby is a placeholder.	screens.js:52-73	Hide until real (\$06 makes them real).
B32	P2	Infinite CSS animations run on hidden elements; keyframes animate <code>filter</code> , <code>box-shadow</code> , <code>text-shadow</code> ; blur over the live canvas.	ui.css:286, 1370, 1881-1919	Gate animations on <code>.show</code> ; animate transform/opacity only.
B33	P2	No <code>-webkit-touch-callout: none</code> ; long-pressing a logo mid-play opens the iOS share sheet.	ui.css	Add it to the stage.

ID	SEV	BUG	WHERE	FIX DIRECTION
B34	P2	Deprecated three.js APIs warn on every boot (THREE.Clock, PCFSoftShadowMap silently downgraded).	renderer.js:76, 271	Move to THREE.Timer and PCFShadowMap .
B35	P2	Switch cinematic (branch): kicker pose stomped by the strafe block; VO and gear toast fire under the beat.	matchScene.js:1139-1141, 4771-4791	Skip strafe while switchRun ; defer VO to openAtBat .
B36	P1	All 12 in-match music files are fragmented MP4; Safari's decodeAudioData rejects the container and the null result is cached, so the city track is silent for the session.	public/assets/audio/music/**; audio.js:8-10, 245-255	Remux to plain m4a (-movflags +faststart) or mp3; log decode failures instead of caching null.
B37	P1	Build target is iOS 16.4+ (class static blocks, private fields, structuredClone); older devices get a blank stage with no message.	vite.config.js; index.html	Set build.target to safari15 or add a boot capability check with an "update iOS" card.
B38	P2	Main renderer requests antialias: true although every frame renders through the composer; the backbuffer MSAA (~25 MB) is never seen.	renderer.js:73	antialias: false on the main renderer.
B39	P2	Theme mp3 carries ID3 cover art (APIC); Safari rejects some tagged MP3s in decodeAudioData , again cached as silence.	public/assets/audio/theme-red-rubber-felony.mp3	Strip tags on export.
B40	P2	TAP IN gate decodes a 2000² PNG logo (16 MB decoded) as the very first thing on a cold start.	branding/logo-square.png; main.js:240	Downsize to 512².
B41	P2	A rejected video.play() is silently converted to muted playback; the second intro clip can lose the gesture window and play silent.	videoPlayer.js:30-34	Log the rejection; reuse one primed element for the set-piece clips.
B42	P2	No visibilitychange handling: after backgrounding, the backdrop videos can stay paused (frozen texture) and audio stays interrupted.	src/ (no handler)	On visible: play() the backdrops, resume audio.
B43	P1	iOS 15–16 enforce a hard canvas memory cap (224–384 MB); sixteen full-size recolour canvases plus their ImageData exceed it and later canvases silently draw nothing (blank or black players).	glbCharacters.js:50-77	Recolour at 1024²; drop the canvas and ImageData after the GPU upload.
B44	P2	GLB source bitmaps are held forever in gltfCache ; on Safari 17+ they are non-purgeable ImageBitmaps (~140 MB for a match's seven 2048² sources).	glbCharacters.js:27, 313-319	Evict source images once cloned.

DEAD WEIGHT AND THE FRAME BUDGET

The project folder carries 628 MB of assets. 162 MB is referenced by nothing. Of what is used, a handful of decisions made for a desktop-class look cost most of the phone's frame: two full-screen videos, a 4× MSAA half-float post chain with bloom, a soft 2048² shadow map, and sixteen un-culled skinned rigs. A quality tier that only trims anti-aliasing cannot rescue any of that.

ASSET WEIGHT BY FOLDER

video (32 mp4)		227 MB
sprites (unused, not deployed)		132 MB
models (21 glb)		73 MB
players (50 png)		64 MB
audio		52 MB
anims (57 mocap glb)		49 MB
textures		16 MB
branding + logos		18 MB

DELETE TODAY, NO BEHAVIOUR CHANGE

public/assets/sprites/ (legacy sheets, run-gen, diverse)	131.5 MB
public/assets/audio/vo/ 28 wav (superseded by announcer packs)	12.2 MB
textures: skyline, asphalt, graffiti-sheet, sky-blacktop png	6.7 MB
branding/logo-poster.png , video/coin-flip.mp4 , audio/match-beat.m4a , logos/bullies-alt.png , arch-band.glb	11.4 MB
spriteCharacters.js , assets.manifest.json , world/blacktop.js , GTAO pass, rapier dependency (zero imports)	~1,300 lines
characters.js (674 lines) ships in the bundle for the ?dance harness only	lazy-load

SHRINK, SAME LOOK

- Seven 2048² PNG atlases → 1024² WebP like the other 14: 43 MB → ~1 MB download, 149 MB → 37 MB GPU.
- Backdrop videos: one per field at 720×960, ≤ 3 Mbps: 227 MB → ~35 MB.
- Portraits to WebP (64 → ~8 MB); music and booth to 96 kbps AAC (52 → ~25 MB).

WHERE THE PHONE'S FRAME GOES (MEASURED LIVE: 157 DRAW CALLS, 862K TRIANGLES, 58 SKINNED MESHES, 35 SHADERS)

COST	TODAY	TIERED TARGET (LOW / HIGH)
Backdrop video	2 × 1248×1664 decoders, VideoTexture upload every frame, mirrored-repeat wrap	poster JPEG only / one 720×960 video
Anti-aliasing + post	4× MSAA HalfFloat ×2 targets, 11-pass bloom, grade, output at 2× DPR	DPR 1.5, no MSAA, no bloom / DPR 2, 2× MSAA, bloom
Shadows	2048² PCFSOft, 16 casters, no frustum culling on any rig	1024² PCF, 6 nearest casters / 2048² PCF
Characters	16 rigs × 18–31k tris drawn in both passes even when off-screen	bounding-sphere culling on both tiers
Per-frame churn	~60–100 short-lived objects per LIVE frame; replay recorder makes 320 KB/s of garbage; HUD DOM rebuilt per frame	scratch vectors, persistent arrays, keyed HUD setters
Behind menus	Full post chain at 60 fps under Title, Menu, Team Select; second renderer in the Locker	render once and pause under DOM screens
Quality watchdog	Steps MSAA 4→2→0 after 8 s, never back up, touches nothing else	boot-time device tier (memory, GPU string, pixels) + watchdog as second guard

"WORKS ON ALL DEVICES" NEEDS A LAB, BUDGETS AND TELEMETRY

Today the game is verified on one Android phone and a software WebKit on Windows. That WebKit harness is valuable (it exposed the lying video decoder and the AudioContext throw) but it cannot decode H.264, has no AudioContext and runs 20× slower than a phone. Nothing has ever measured the game on a real iPhone, an iPad, a Chromebook, or a \$150 Android.

COVERAGE TODAY

SURFACE	UNIT TESTS	SCRIPTED PLAYTHROUGH	REAL DEVICE	TELEMETRY
Android Chrome (dev's phone)				
iPhone Safari / PWA		software WebKit, no video, no audio	testers only	
iPad				
Low-end Android (2–4 GB)				
Desktop Chrome / Edge		headless renders black		

covered partial known broken none

THE DEVICE LAB (BUY OR RENT)

DEVICE	WHY
iPhone SE (2nd/3rd gen), 3–4 GB	Smallest screen, lowest iOS memory ceiling; the phone that reloads first
iPhone 11 or XR	The most common 4 GB iPhone still in use
iPhone 14/15	Current baseline; DPR 3 and ProMotion timing
iPad (9th/10th gen)	Letterboxed portrait frame, different safe areas, tablet App Store screenshots
Galaxy A14/A15, Moto G	Mali GPU, 4 GB, the Android install base outside the US
Pixel 6a/7a	Mid-tier Adreno/Mali reference

Rent the rest through BrowserStack App Live or AWS Device Farm for release checks. Xcode's Simulator does not run WebGL like a device; only real hardware counts for memory.

BUDGETS TO ENFORCE (PER RELEASE)

METRIC	GATE
Frame time, LIVE play, iPhone 11 / Galaxy A15	≤ 20 ms p95 (50 fps floor)
Web content memory, iPhone SE, match 3	≤ 450 MB
Tap-to-first-pitch, warm cache	≤ 8 s
Main-thread block during match build	≤ 100 ms any single task
Installed app size / first-run download	≤ 150 MB / ≤ 120 MB
Crash-free sessions (telemetry)	≥ 99.5 %
Stalled games (no phase change 30 s)	0 per 1,000 games

AUTOMATION TO ADD

- A game-to-the-end harness (walk-off, final out, walk-off homer) in `round-e2e.mjs`. B01 would have failed it.
- A phase-age watchdog in the game plus a "stalled game" telemetry event with the phase, last timer and last error.
- Memory and frame-time sampling on device via the Capacitor shell (native memory API), reported per match.
- Playwright WebKit runs stay in CI for logic; real-device runs on every release candidate.

THE STORE PATH IS A WRAPPER, AN ASSET DIET, AND RAILS

The original assumption holds: this game can be wrapped for the App Store and Google Play without a rewrite. Capacitor puts the existing Vite build inside a native shell (WKWebView on iOS, Android WebView) and exposes native APIs through plugins. Apple accepts packaged HTML5 games when they behave like apps, not websites. What Apple and Google will reject is a 500 MB install, a subscription sold outside in-app purchase, and an app that hangs at the final out.

THE SHELL

- **Capacitor 7** with the Vite build as the web dir; plugins: App (lifecycle, back button), Preferences (save, replacing localStorage), Haptics, Screen Orientation lock, Status Bar, Splash Screen, Filesystem (downloaded packs), Network.
- **iOS specifics:** `allowsInlineMediaPlayback` for the backdrop and intro videos, audio session category *ambient* so the game respects the mute switch, background audio off, `limitsNavigationsToAppBoundDomains`.
- **Android specifics:** hardware acceleration on the WebView, target API 35, 64-bit only, AAB with Play Asset Delivery or on-demand packs.
- **Build machines:** iOS needs Xcode on a Mac. A used Mac mini or a macOS cloud runner both work.

THE ASSET DIET (MANDATORY)

- **Ship in the binary:** UI, engine, two default crews, the Blacktop, SFX, one booth voice. Target ≤ 80 MB.
- **Download on demand:** city packs (field video, crew models, music, booth lines) from a CDN into the Filesystem with a progress card, verified by hash. Target ≤ 12 MB per city after re-encoding.
- Apple's App Store limit is 4 GB and cellular downloads over 200 MB prompt the user; Google's base AAB is 200 MB. Neither is the real constraint. Memory is: a 500 MB install is a 500 MB working set on a phone that reloads at 900 MB.

THE RAILS A SUBSCRIPTION NEEDS

- **Purchases:** RevenueCat over StoreKit 2 and Google Play Billing. Handles receipts, renewals, grace periods, restore, and gives one entitlement flag the game reads. Apple rule 3.1.1: digital goods and subscriptions must go through IAP; no Stripe links in-app.
- **Accounts:** Supabase Auth with Sign in with Apple and Google. Apple requires Sign in with Apple whenever another social login is offered, and in-app account deletion (5.1.1 v).
- **Cloud save:** the existing `SaveManager` already abstracts the backend; sync the same JSON blob to Postgres with a version stamp. Required for a subscription to survive a phone change.
- **Telemetry:** Sentry (crashes, stalled-game events, release health) and PostHog or Firebase Analytics (retention, funnel, session length). Both have free tiers.
- **Legal:** privacy policy and terms of service URLs (required on the store page and inside the app for auto-renewing subscriptions), subscription disclosure copy, age rating (likely 4+ / Everyone; the copy has no profanity), App Privacy labels, Android Data Safety form, iOS Privacy Manifest for required-reason APIs.

STORE SUBMISSION CHECKLIST

- | | | |
|---|--|---|
| ☐ Apple Developer Program (\$99/yr) and App Store Connect app record | ☐ Icons, launch screen; 6.7", 6.5", 5.5" iPhone screenshots + iPad set; Play feature graphic; preview video (Higgsfield pipeline) | ☐ Age rating questionnaires (App Store, IARC) |
| ☐ Google Play Console (\$25 once); new personal accounts must run a closed test with 12+ testers for 14 days before production | ☐ Subscription products, groups, localized descriptions, intro offers; Restore Purchases button | ☐ Sign in with Apple; account deletion flow; export data on request |
| ☐ Bundle IDs, signing certificates, provisioning; Play app signing | ☐ Privacy policy, terms, support URL; privacy labels; Data Safety; Privacy Manifest | ☐ TestFlight external beta and Play closed testing complete |
| | | ☐ Review notes with a demo account and a "how to reach the end of a game" script |

WHAT MAKES PEOPLE COME BACK EVERY DAY, AND PAY EVERY MONTH

Apple and Google both require a subscription to deliver ongoing value, and players require it to feel fair. The rule of thumb that survives review and community: sell content, cosmetics, convenience and status, never stats. Today the Locker's cleats give +6–12 % speed; that stays earnable, not purchasable. The game already has the bones (Locker, crews, trophies, tutorial, replays, city elements, a dance bag). What it lacks is a reason to open it tomorrow.

REAL DAILY AND WEEKLY CHALLENGES

CORE

The daily card exists but is hardcoded to "0/3". Make it real: three daily goals, one weekly, streak multiplier, a reward chest of crowns and gear.

Why: the single biggest lever on day-1 and day-7 retention in mobile sports games.

SEASON LADDER

CORE

Six-week seasons. Placement matches, tiers from Rookie to King of the Streets, promotion and relegation, end-of-season kit drop. Works offline against crews first, online later.

Why: a reason the month matters; the natural anchor for a monthly Pass.

CREW CAREER

CORE

Run the Map becomes a season schedule: home and away, standings, rivalries with heat carried over, playoffs, and The Crown final at your rival's yard.

Why: turns ten one-off matches into a 20-hour arc. The trophy system already supports it.

CREATE-A-KICKER

CORE

Twenty archetypes, skin tones, hair, bands, kits and numbers already exist in the character pipeline. Let the player build their captain, name them, and grow their stats through play.

Why: identity is what people pay to dress.

DERBY

QUICK PLAY

The menu already promises it. A 90-second home-run derby with the crown meter, leaderboards, and a daily wind. Short sessions for the bus and the bathroom.

Why: session count, not session length, is what daily habits are built on.

GHOST RIVALS (ASYNC PVP)

SOCIAL

Play against a friend's recorded kicking and pitching profile; challenge links; weekly rival. Real-time online can come later; kickball is turn-based enough that async feels like a duel.

Why: social play without servers or matchmaking risk on day one.

LEADERBOARDS AND CLUBS

SOCIAL

Global, friends and city boards for Derby and the Ladder. Clubs of real players with a shared crew name and weekly club goals.

Why: status is a subscription feature people show off.

MONTHLY CITY DROP

CONTENT

One new yard, crew, city track and booth pack per month using the existing Higgsfield pipeline. Pass members get it a week early with an exclusive kit.

Why: a live-ops calendar is what "ongoing value" means to Apple's reviewers and to players.

HIGHLIGHT EXPORT

GROWTH

The replay recorder already captures every play. Render a 10-second vertical clip with the booth call and share it to TikTok and Instagram; watermark on free, clean on Pass.

Why: the cheapest acquisition channel a sports game has.

WEEKEND BLOCK PARTY EVENTS

CONTENT

Rule mutators built from the city elements: night court, sea breeze, train rumble, double crowns. Rotating, time-boxed, with an event kit.

Why: novelty without new content; a reason to open the app on Saturday.

FREE VS KINGS PASS

A fair split: the free game is the whole game. The Pass buys time, content, identity and status. Suggested price points: \$4.99 per month, \$29.99 per year, with a 7-day trial; a one-time Founder's kit at launch for early supporters. Optional rewarded video ads on free (crowns for a 30-second ad) are the second revenue line and never interrupt play.

FREE The full game, forever ■ Every mode: 1v1, Crew Career, Derby, Ladder, tutorial drills ■ Three crews and their yards unlocked, one rotating free crew each week ■ Daily and weekly challenges, streaks, the base reward track ■ Create-a-Kicker with the starter wardrobe ■ All stat gear (cleats, kicks) earned through play, never sold ■ Leaderboards, clubs (join), ghost rivals ■ Highlight export with a watermark ■ Rewarded ads for crowns, opt-in only	KINGS PASS \$4.99 / month · \$29.99 / year · 7-day trial ■ All ten crews and yards, plus every monthly city drop a week early ■ The Season Pass cosmetic track: exclusive kits, cleats colourways, taunts, dances, ball skins ■ Double XP and a second daily challenge slot ■ Create a club, custom crew crest and name in the booth call ■ Clean highlight export, longer clips, booth voice choice ■ Cloud save across devices (free accounts sync too; Pass adds unlimited kicker slots) ■ Event kits for Block Party weekends ■ King badge on leaderboards and rival cards
--	---

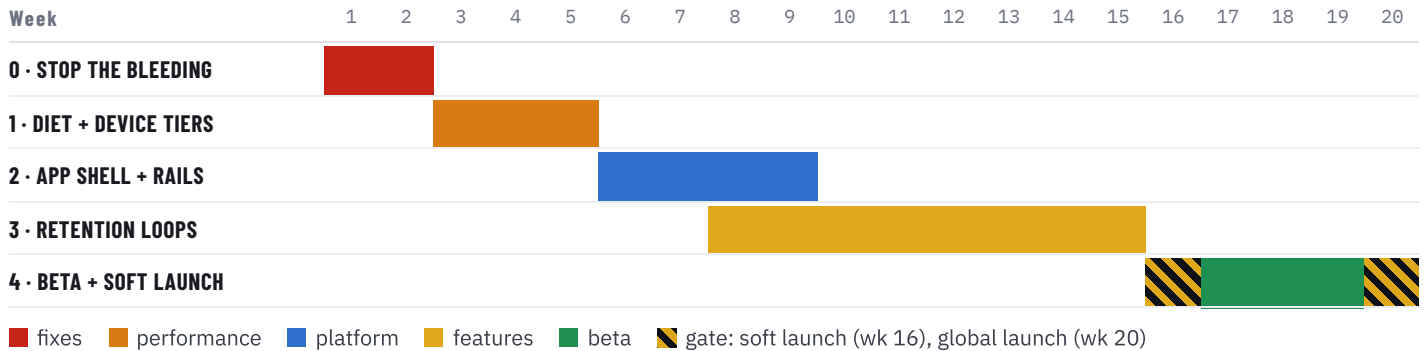
RULES THAT KEEP THE PASS APPROVABLE AND THE GAME RESPECTED • No stat for money. Speed, power and steal bonuses stay earned. Cosmetic-only Pass items keep the Ladder honest and keep app-review and community goodwill. • Clear price, term, and auto-renew text before the buy button; a working Restore Purchases; cancellation through the store, never a support email. • Every Pass benefit must be visible to free players (the kit on the field, the badge on the board). Invisible value does not sell. • Ship the Ladder and the daily loop before the Pass. A subscription launched into a game with no reason to return is a refund queue.
--

LAUNCH GATES (SOFT LAUNCH, TWO COUNTRIES)

METRIC	GATE TO GLOBAL LAUNCH	SOURCE
Crash-free sessions	≥ 99.5 %	Sentry
Day-1 / Day-7 retention	≥ 35 % / ≥ 12 %	PostHog
Median session	≥ 8 minutes, ≥ 2 sessions per day for retained users	PostHog
Trial start rate / trial-to-paid	≥ 6 % of day-3 users / ≥ 40 %	RevenueCat
Store rating	≥ 4.3 with in-app review prompt after a won Ladder match	App Store / Play

FROM HERE TO LAUNCH

Order matters. Nothing ships to a store, and no new feature is worth building, until the game reliably reaches its own final out on an iPhone SE. Phase 0 is two weeks of fixes that make the current game trustworthy; Phase 1 makes it light; Phase 2 gives it a body in the stores; Phase 3 gives it a reason to be opened daily; Phase 4 proves it with strangers before the world sees it.



PHASE 0 · WK 1-2 B01–B14 in ledger order, hoisted nets, field dispose, context-loss handling, global error handler and Sentry, a game-to-the-end harness. Verified on a real iPhone SE and iPhone 11 before merge. Ship to production; this alone ends the "freezes on Apple" reports.

PHASE 1 · WK 3-5 Delete 162 MB; re-encode videos and atlases; device tier at boot (DPR, MSAA, bloom, shadows, video vs poster); stop rendering behind menus; chunk the character build; code-split; audio eviction. Gate: memory $\leq$ 450 MB on iPhone SE at match 3, 50 fps floor on Galaxy A15.

PHASE 2 · WK 6-9 Capacitor iOS + Android shells, on-demand city packs from a CDN, Supabase accounts and cloud save, RevenueCat products, legal pages, privacy labels, store listings, TestFlight and Play internal builds, the device lab.

PHASE 3 · WK 8-15 Daily/weekly challenges and streaks (real), Season Ladder, Derby, Create-a-Kicker, leaderboards and clubs, Pass cosmetic track, highlight export, the first monthly city drop through the pipeline. Ladder and daily loop ship first.

PHASE 4 · WK 16-20 Closed beta (satisfies Google's 14-day rule), balance from telemetry, ASO assets, soft launch in two countries, KPI gates from \$07, then global launch and a live-ops calendar with a drop every month.

HOW THIS WAS MEASURED

VERIFIED DIRECTLY

- Static read of every module in `src/` (67 live files, 21,151 lines) plus the uncommitted branch diff; import-graph walk from `main.js`.
- Git history: the game-over clause traced to 9429882, contained in `main` and in the production deploy.
- `vitest run`: 65 files, 688 tests, all passing in 8.4 s.
`vite build`: 1.19 MB JS (326 KB gzip), 63 KB CSS, one code-split chunk.
- Two live probes against production with Playwright WebKit in an iPhone 13 profile: console and page errors (none), frame cadence (2–3.6 s gaps, one 14.4 s gap during build), 41–54 s to a live phase under software rendering.
- GPU inventory in a live match: 60 unique textures totalling 242–267 MB, 54 of them canvases, seven at 2048²; render targets 27 MB (MSAA off) to 69 MB (4×); 2048² shadow map; 157 draw calls; 861,672 triangles; 58 skinned meshes; 16 shadow casters; 35 shader programs; two backdrop videos created per field.
- Container parsing of every backdrop mp4: 1248×1664 H.264, 10 s, 3–14.6 Mbps; intros 720×1280.
- Asset inventory: `du` and reference expansion of every asset-path template in the source.

ESTIMATED, AND HOW

- Video decoder memory: two 1248×1664 H.264 decoders with 3–6 YUV surfaces each plus two RGBA video textures, ~40–80 MB. Not measurable from JavaScript.
- Decoded audio: PCM size from track length at 44.1 kHz stereo float32; a 2–3 minute track is 40–60 MB. Not measured on device.
- JS heap and canvas backing stores: from rig triangle counts, mocap pack sizes and the recolour cache design. WebKit exposes no heap counter.
- iOS kill thresholds: from WebKit behaviour on 3–4 GB devices; Apple publishes no figure.
- Timeline: assumes one experienced engineer plus the existing AI-assisted workflow, no new art pipeline, and the current scope of ten crews.

NOT COVERED HERE

- Real-device profiling. Every number above should be re-measured on an iPhone SE in Phase 0 with Safari Web Inspector's timelines and the Capacitor shell's memory API.
- Gameplay balance (homer rate, rob-call frequency, park size) noted as open in the session log.
- Backend architecture for real-time PvP; async ghosts are recommended first for that reason.

VERIFIED WORKING (DO NOT RE-AUDIT)

- rAF loop isolates every frame callback, timer and bus listener in try/catch
- AudioContext construction is try/caught; all callers null-guard
- 688 unit tests pass; WebKit end-to-end harnesses run against production
- rAF safety pump re-arms a dropped frame (WebKit)
- Video backdrop poster-first swap with pixel watchdog (July hardening)
- Save system tolerates private mode and quota errors; per-character clones are disposed
- Timer queue splice bug from July is fixed; timers run on real time and pause with the engine
- RunnerWatchdog runs phase-independently above the phase blocks

FILES MOST WORTH A FIRST READ

<code>src/game/matchScene.js</code>	5,019 lines; the match orchestrator, every phase and watchdog. B01, B04, B06–B09, B20, B22, B25–B27 live here.
<code>src/engine/renderer.js</code>	Post chain, pixel ratio, MSAA watchdog, no context-loss handling.
<code>src/game/field.js</code>	Two videos per field, shadow map, no dispose.
<code>src/game/glbCharacters.js</code>	Recolour cache, atlas walks, un-culled shadow casters.
<code>src/engine/audio.js</code>	Unbounded buffer cache, music race, missing interrupted-state handling.
<code>src/main.js</code>	Boot and match flow with no error path; service-worker registration.